

Public Key Infrastructure and the Full Picture

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



August 7, 2026

Table of Content

1 Revisiting the Full Picture

2 Discussion on Important Points

Recommended Reading From textbook: Chapter *

Revisiting the Full Picture of Secure Communication

Secure Communication: All four phases

1 Setting Up Trust

- A trusted Certificate Authority (CA) establishes the root of trust.
- Each party generates a public/private key pair.
- The CA verifies the party's identity and issues a digital certificate binding the identity to its public key.

2 Establishing Trust Between Parties

- Parties exchange their certificates over the public channel.
- Each party verifies the CA's signature using a trusted CA public key.
- This authenticates the public keys and prevents Man-in-the-Middle attacks.

3 Establishing a Secure Session

- Parties perform an authenticated key exchange such as DH/ECDH.
- Both derive the same shared secret without transmitting it.
- A KDF converts the shared secret into suitable session keys.

4 Secure Communication

- Symmetric encryption provides efficient confidentiality for long messages.
- AEAD provides confidentiality, integrity, and authentication; AAD is authenticated but not encrypted.
- A fresh nonce is used for each message; keys may be refreshed periodically and discarded when the session ends.

Phase 1: CA Establishes the Root of Trust

The Certificate Authority (CA) first generates its own key pair

$$(pk_{CA}, sk_{CA}).$$

The CA creates a root certificate containing its public key and identity.

$$\text{Root Certificate} = \text{Identity}_{CA} \parallel pk_{CA} \parallel \text{Validity} \parallel \dots$$

The root certificate is distributed through a trusted mechanism and the CA public key becomes a

Root of Trust.

Examples of trust stores: Operating Systems Browsers Applications

Informally, CAs are the organizations from which we purchase domains, such as Let's Encrypt (ISRG), GlobalSign, Sectigo, and GoDaddy.

Phase 1: Party Generates Its Key Pair

Suppose party A wants to communicate securely.
 A generates

$$(pk_A, sk_A) \leftarrow \text{KeyGen}(1^\lambda).$$

- pk_A is the public key and may be distributed.
- sk_A is the secret key and must remain under A 's control.
- The secret key is never sent to the CA or to other parties.

A now requests a certificate binding

$$\text{Identity}_A \longleftrightarrow pk_A.$$

Phase 1: Certificate Signing Request and Issuance

A sends a Certificate Signing Request (CSR) containing

$(\text{Identity}_A, pk_A, \text{additional information})$

to the CA.

The CA verifies that A controls the claimed identity.

The CA then creates and signs a certificate

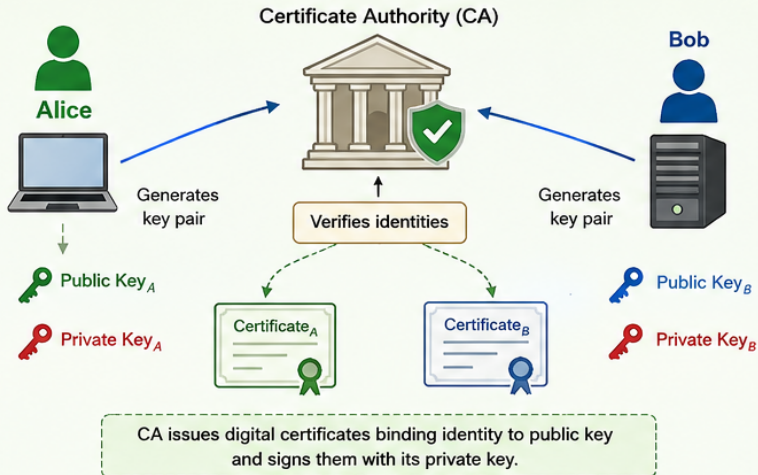
$$\text{Cert}_A = \text{Sign}_{sk_{CA}} (\text{Identity}_A \parallel pk_A \parallel \text{Validity} \parallel \dots).$$

The CA returns Cert_A to A.

Important

The CA does not need to know sk_A . A proves possession of the corresponding secret key during the certificate enrollment process.

1. Setting Up Trust



Phase 2: Certificate Exchange

When A initiates communication with B , A presents

$\boxed{Cert_A}$

to B .

The certificate contains

(Identity _{A} , pk_A , Validity, CA signature.)

B already trusts the CA public key

pk_{CA} .

Therefore, B can verify

$$\text{Verify}_{pk_{CA}}(Cert_A) = 1.$$

$\boxed{\text{Certificate} \implies \text{Authenticated Public Key}}$

Phase 2: Preventing the Man-in-the-Middle Attack

Without certificate verification, an attacker could replace

$$pk_A$$

with the attacker's public key

$$pk_E.$$

The certificate prevents this because the attacker cannot modify

$$\text{Identity}_A \parallel pk_A$$

without forging the CA's signature.

B accepts pk_A only if

$$\text{Verify}_{pk_{CA}}(\text{Identity}_A \parallel pk_A, \sigma_{CA}) = 1.$$

Result

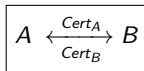
The public key is not merely public; it is **authenticated**.

Phase 2: Mutual Authentication

If both parties need authentication, B also possesses

$$(pk_B, sk_B, Cert_B).$$

Then both parties verify each other's certificates.

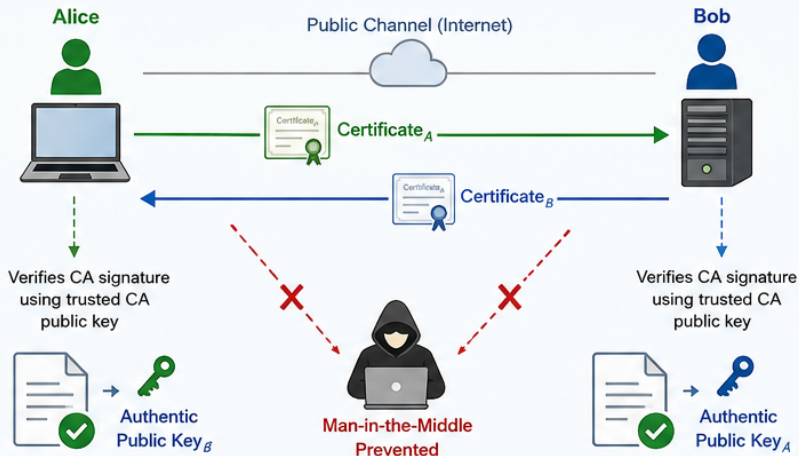


Thus,

A authenticates B and B authenticates A .

This is commonly called
mutual authentication.

2. Establishing Trust Between Parties



Phase 3: Key Exchange

After authentication, A and B establish a shared secret.

For Diffie–Hellman:

$$a, b \xleftarrow{\$} \mathbb{Z}_q$$

and compute

$$A = g^a, \quad B = g^b.$$

They exchange A and B over the public channel.

$$K = B^a = A^b = g^{ab}.$$

The values g^a and g^b do **not** need to be encrypted.

They need to be **authenticated** so that an attacker cannot replace them.

Phase 3: Authentication of the Key Exchange

The ephemeral key-exchange values must be bound to the authenticated identities. Conceptually,

$$A = g^a, \quad \sigma_A = \text{Sign}_{sk_A}(A \parallel \text{context}),$$

and similarly

$$B = g^b, \quad \sigma_B = \text{Sign}_{sk_B}(B \parallel \text{context}).$$

The receiver verifies the signatures using the already authenticated public keys.

Authenticated Public Keys + Authenticated DH Values

$\Rightarrow$ Authenticated Key Establishment

Phase 3: From Shared Secret to Session Keys

The DH shared secret

$$K = g^{ab}$$

is generally not used directly as an encryption key.

Both parties apply a Key Derivation Function (KDF):

$$K_{\text{session}} = \text{KDF}(K, \text{context})$$

The KDF can derive multiple independent keys:

$$K \longrightarrow \begin{cases} K_{\text{enc}} \\ K_{\text{auth}} \\ K_{\text{other}} \end{cases}$$

The resulting keys have the required length and are suitable for symmetric cryptographic algorithms.

Phase 3: Secure Session Established

After key derivation,

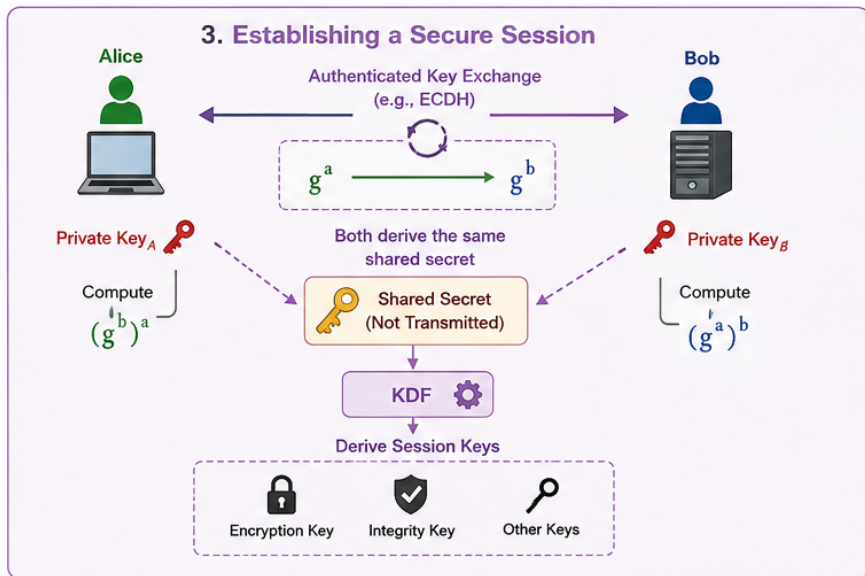
Public-Key Cryptography $\longrightarrow$ Shared Secret $\longrightarrow$ KDF $\longrightarrow$ Session Keys

The expensive public-key operations are now finished.

For subsequent communication, A and B use efficient symmetric cryptography (Block Cipher / Stream Cipher).

Session Key $\longrightarrow$ AES-GCM / ChaCha20-Poly1305

Phase 3: Summary



Phase 4: Session Key to Secure Communication

For every message M_i , a fresh nonce N_i is selected.

An AEAD scheme computes

$$(C_i, T_i) = \text{Enc}_{K_{\text{session}}}(N_i, A_i, M_i).$$

The transmitted data is conceptually

$$N_i \parallel A_i \parallel C_i \parallel T_i.$$

where

- N_i : nonce,
- A_i : associated data,
- C_i : ciphertext,
- T_i : authentication tag.

The nonce is normally public; its critical requirement is appropriate uniqueness under the relevant key.

Phase 4: Decryption and Verification

The receiver obtains

$$(N, A, C, T).$$

The AEAD algorithm first verifies the authentication tag.

$$\text{Verify}(K, N, A, C, T)$$

If verification fails,

Reject

and the ciphertext must not be trusted.

If verification succeeds,

$$M = \text{Dec}_K(N, A, C, T).$$

Thus AEAD provides

Confidentiality + Integrity + Authentication.

AAD is authenticated but remains unencrypted.

Phase 4: Rekeying and Session Termination

A long-lived session may periodically refresh its keys.

$$K_1 \longrightarrow K_2 \longrightarrow K_3 \longrightarrow \dots$$

Rekeying limits the amount of data protected under a single key and can provide additional security properties.

When communication terminates,

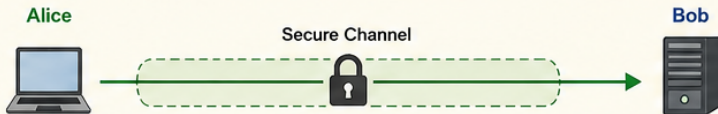
Session Keys $\longrightarrow$ Discarded

For ephemeral Diffie–Hellman, destruction of ephemeral secret values also supports

Perfect Forward Secrecy (PFS)

when the protocol is designed appropriately.

4. Secure Communication



Fresh Nonce

A unique nonce/IV is used for each message.



Key Refresh

Keys may be refreshed periodically.



End of Session

Keys are discarded when the session ends.

Provides Confidentiality, Integrity, and Authentication

Fundamental Cryptographic Primitives

Primitive	Purpose	Examples
Block Cipher	Confidentiality for fixed-size blocks; used with modes of operation for long messages	AES, SM4
Stream Cipher	Confidentiality by generating a pseudorandom keystream	ChaCha20, Salsa20
Hash Function	Fixed-length digest for integrity and fingerprinting	SHA-256, SHA-3
Public-Key Encryption	Securely transmit secrets without a pre-shared key	RSA, ElGamal, ML-KEM
Digital Signature	Authentication, integrity, and non-repudiation	DSA, ECDSA, ML-DSA

Observation

Modern cryptographic protocols are built by combining these primitives. For example,

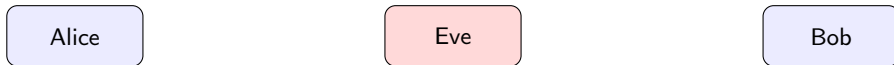
Public-Key Encryption → Key Exchange → Session Key → Block/Stream Cipher + AEAD.

Man-in-the-Middle (MITM) Attack

The need of Certificate of Public Key.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.

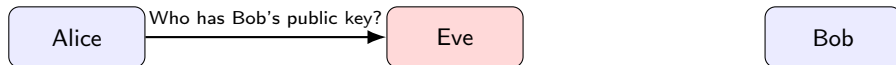


Alice wants to communicate securely with Bob.

Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Alice asks for Bob's public key.

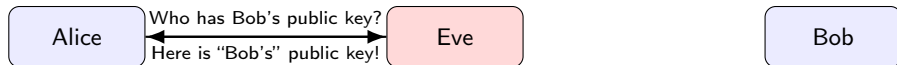
Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Eve intercepts the request and sends her own public key.

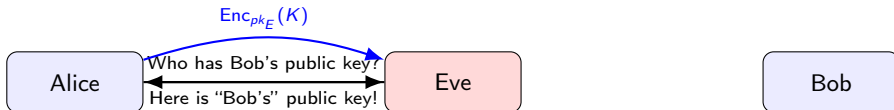
Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Alice unknowingly encrypts the session key using Eve's public key.

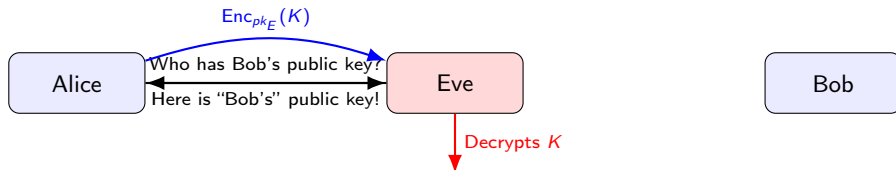
Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Eve decrypts the session key and learns the secret.

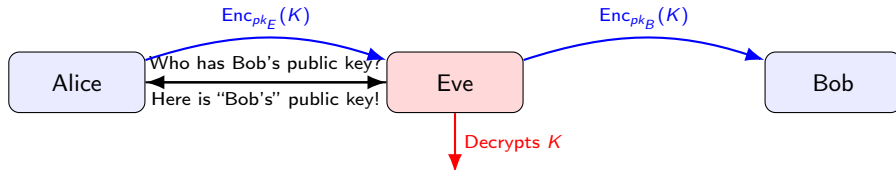
Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Why Must Public Keys Be Authenticated?

Suppose Alice wants to establish a shared secret with Bob.



Eve re-encrypts the same session key using Bob's public key and forwards it. Bob believes the key came directly from Alice.

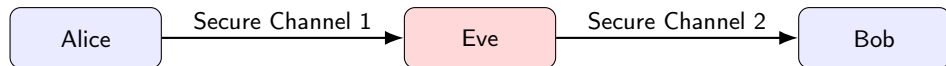
Alice mistakenly encrypts her secret using Eve's public key instead of Bob's.

Result

Eve can decrypt the message, even though Alice believes she is communicating securely with Bob.

Man-in-the-Middle (MITM) Attack

Without authenticated public keys, an attacker can establish two independent secure channels.



Eve decrypts every message, reads or modifies it, re-encrypts it, and forwards it to Bob. Neither Alice nor Bob realizes that the communication is being intercepted.

Solution

Authenticate public keys using digital certificates and a trusted Public Key Infrastructure (PKI).

Towards Public Key Infrastructure (PKI)

Public keys themselves must be authenticated before they can be trusted.

This is achieved using

- Digital Certificates,
- Certificate Authorities (CA),
- Public Key Infrastructure (PKI),
- Certificate Chains.

Trust the Certificate $\implies$ Trust the Public Key

Discussion on Important Points

- 1 Hash Function
- 2 Symmetric Key encryption
- 3 Public-key Encryption
- 4 Digital Signature

Hash Function

Where Do Hash Functions Appear?

We have mainly seen two kinds of hash functions:

Cryptographic Hash Function

Universal Hash Function

Hash functions are often not directly visible in the communication flow, but are used internally in several cryptographic mechanisms.

- 1 **Digital Signatures:** A cryptographic hash compresses a long message before signing.

$$m \longrightarrow H(m) \longrightarrow \text{Sign}_{sk}(H(m)).$$

- 2 **Key Derivation:** Cryptographic hash functions are used inside KDFs to derive session keys. For example:

- **HKDF-SHA-256:** uses SHA-256 inside HKDF to derive keys.
- **TLS 1.3:** uses HKDF with SHA-256/SHA-384 to derive handshake and application traffic keys.

- 3 **AEAD Authentication:** A keyed universal hash is used inside constructions such as AES-GCM for authentication.

Hash Functions in the Complete Communication Flow

Hash functions appear at different stages of the system.

Phase	Use	Hash Type
Trust / Certificates	Hash data before digital signing	Cryptographic hash
Key Establishment	Hashing may be used in the authenticated handshake	Cryptographic hash
Key Derivation	Derive session keys from shared secret	Cryptographic hash
Secure Communication	Authentication tag in AEAD like AES-GCM	Universal hash (GHASH)

Big Picture

A hash function is a **building block**, not a security service by itself. Its security role depends on the construction in which it is used.

Symmetric Key Encryption

Symmetric-Key Encryption for Secure Communication

After key establishment and key derivation, the resulting session keys are used for efficient communication.

- **Confidentiality:** For long messages, we use either a **stream cipher** or a **block cipher with a suitable mode of operation**.

$$M \xrightarrow{K} C$$

- **Fundamental Primitives:** Block and stream ciphers are fundamental symmetric-key primitives, designed to provide efficient confidentiality using a shared secret key.
- **Authentication and Integrity:** Encryption alone does not detect modification of the ciphertext. Therefore, an authentication mechanism generates a short **authentication tag** that is transmitted with the ciphertext.

$$(C, T)$$

- **Modern Practice:** Instead of using separate encryption and authentication mechanisms, we commonly use **AEAD** constructions such as AES-GCM or ChaCha20-Poly1305.

Key Idea

Session Key $\rightarrow$ AEAD $\Rightarrow$ Ciphertext + Authentication Tag

Authentication

Authentication: Before and After Key Establishment

Authentication is needed at two different stages.

1 Before a Shared Secret Exists: Digital Signature

A party uses its private key to authenticate a message or handshake transcript:

$$\sigma = \text{Sign}_{sk}(m).$$

Anyone with the authenticated public key can verify:

$$\text{Verify}_{pk}(m, \sigma) = 1.$$

2 After a Shared Secret Exists: Message Authentication Code

Once Alice and Bob share a secret key K , they can authenticate messages using a keyed authentication mechanism:

$$T = \text{MAC}_K(m).$$

The receiver recomputes the tag and accepts only if it matches.

Key Difference

Digital signatures use a **private/public key pair**; MACs use a **shared secret key**. A MAC provides authentication and integrity, but not public verifiability.

Keyed Hash and MAC Constructions

A MAC can be constructed using different underlying primitives.

- **HMAC:** Uses a cryptographic hash function.

$$\text{HMAC}_K(m) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel m)).$$

- **CMAC:** Uses a block cipher, typically AES, to construct a secure MAC.

$$T = \text{CMAC}_K(m).$$

- **Keyed Universal Hash:** Uses a secret hash key and a universal hash construction.

$$T = \text{UH}_K(m).$$

A well-designed construction can provide efficient authentication with a short tag.

Examples

HMAC-SHA-256

AES-CMAC

GHASH / Poly1305

Modern Practice

In AEAD schemes such as AES-GCM and ChaCha20-Poly1305, authentication is integrated with encryption, so a separate MAC is normally not needed.

Where Are MACs Still Used with AEAD?

AEAD already provides encryption, integrity, and message authentication, so a separate MAC is normally unnecessary for the encrypted data.

However, MAC constructions such as HMAC and CMAC are still useful in other settings.

- **Authentication without Encryption:** When confidentiality is not required, a MAC can authenticate a message efficiently.

$$T = \text{MAC}_K(m).$$

- **Key Derivation:** HMAC is used as a building block in KDFs such as HKDF.
- **Protocol Authentication:** Some protocols use MACs to authenticate control messages, session information, or protocol state.
- **Specialized Constructions:** CMAC and keyed universal hashes can be used where their particular performance or construction properties are desirable.

Key Point

For ordinary encrypted application data, AEAD replaces the need for a separate encryption + MAC construction.



Public Key Encryption

Public-Key Cryptography: Where Is It Used?

Public-key cryptography uses a key pair

$$(pk, sk),$$

where pk can be publicly distributed while sk remains secret.

- **Digital Signatures:** The secret key is used to generate a signature, while the public key is used for verification.

$$\sigma = \text{Sign}_{sk}(m), \quad \text{Verify}_{pk}(m, \sigma).$$

- **Key Establishment:** Public-key techniques allow parties to establish a shared secret over an insecure channel. Examples include DH/ECDH and public-key KEMs such as ML-KEM.
- **Public-Key Encryption:** Anyone can encrypt using the recipient's public key, while only the recipient can decrypt using the corresponding secret key.

$$c = \text{Enc}_{pk}(m), \quad m = \text{Dec}_{sk}(c).$$

- **Authentication and Trust:** Public keys are used to verify digital certificates and establish authenticated identities in PKI.

Key Observation

Public-key cryptography is computationally more expensive than symmetric cryptography and is therefore mainly used for **authentication and key establishment**, not bulk data encryption.

Popular Public-Key Encryption Schemes

Public-key encryption schemes have been constructed from different mathematical hardness assumptions.

Scheme	Hardness Assumption	Type
RSA-OAEP	Integer Factorization	Classical
ElGamal	Discrete Logarithm (DLP)	Classical
ECIES	Elliptic Curve DLP	Classical
Paillier	Composite Residuosity	Classical
McEliece	Syndrome Decoding	Post-Quantum
NTRU	Lattice Problems	Post-Quantum
LWE-based PKE	Learning With Errors	Post-Quantum
ML-KEM (Kyber)	Module-LWE	Post-Quantum KEM

Modern Direction

Classical PKE based on RSA and DLP is vulnerable to quantum attacks. Modern key establishment is moving toward post-quantum KEMs such as **ML-KEM (Kyber)**.

We will study some of the above.

Key Agreement

Key Exchange: One-Sided vs. Two-Sided Contribution

Earlier key-establishment methods based on public-key encryption can have a **one-sided contribution**.

PKE-Based Key Transport

Alice chooses the session secret

$$K \xleftarrow{\$} \{0,1\}^\ell.$$

She encrypts it using Bob's public key:

$$C = \text{Enc}_{pk_B}(K).$$

Bob decrypts and obtains K .

$$K_A = K$$

Only Alice contributes the secret randomness.

Key Distinction

PKE-based key transport: **one party chooses the secret**. It is no more used.

DHKE: **both parties contribute to the shared secret**.

Why Does DHKE Work?

Correctness follows from

$$(B)^a = (g^b)^a = g^{ab} = (g^a)^b = (A)^b.$$

Security Assumption

Given

$$(g, g^a, g^b),$$

an adversary should not be able to compute

$$g^{ab}.$$

This is known as the **Computational Diffie-Hellman (CDH) Assumption**.

The underlying hardness comes from the Discrete Logarithm Problem (DLP).

Choosing the Group

The security of DHKE depends entirely on the choice of the group.

Group	Assumption
$\mathbb{Z}_p^*$	DLP
Elliptic Curve Groups	ECDLP
Class Groups	Class Group DLP

The group should satisfy

- Efficient group operations.
- Efficient exponentiation.
- Computationally hard Discrete Logarithm Problem.
- No known efficient classical attacks.

Typical choices include

- Multiplicative group $\mathbb{Z}_p^*$,
- Elliptic curve groups.

Observation

The same protocol can be instantiated over different algebraic groups, provided the DLP remains computationally hard.

Examples of Classical Key Agreement Protocols

Most classical key exchange protocols rely on the hardness of the Discrete Logarithm Problem.

Protocol	Underlying Assumption
Diffie-Hellman (DH)	DLP / CDH
Elliptic Curve DH (ECDH)	Elliptic Curve DLP
MQV / HMQV	DLP
SIGMA	DLP + Digital Signatures
TLS 1.2 (ECDHE)	Elliptic Curve DLP

Other key establishment mechanisms are based on different assumptions.

RSA $\rightarrow$ Integer Factorization

The Quantum Challenge

Classical key agreement protocols rely on mathematical assumptions such as

- Discrete Logarithm Problem,
- Integer Factorization.

Large-scale quantum computers can solve both problems efficiently using

Shor's Algorithm.

Consequently,

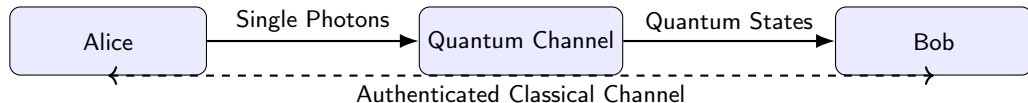
- Diffie-Hellman becomes insecure,
- ECDH becomes insecure,
- RSA key exchange becomes insecure.

Next Step

Can we design key agreement protocols that remain secure even against quantum computers?

Solution 1: Quantum Key Distribution (QKD)

Instead of relying on computational hardness assumptions, Quantum Key Distribution (QKD) derives its security from the laws of quantum mechanics.



Idea

Alice encodes random bits into quantum states. Bob measures them. Any eavesdropping disturbs the quantum states and is therefore detectable.

Limitations of QKD

Although QKD provides information-theoretic security, practical deployment remains challenging.

- Requires dedicated quantum communication channels.
- Requires specialized quantum hardware.
- Limited transmission distance without quantum repeaters.
- Expensive deployment and maintenance.
- Still requires an authenticated classical communication channel.

Observation

QKD is suitable for specialized high-security applications, but is currently impractical for securing the global Internet.

Solution 2: Post-Quantum Key Exchange

Instead of changing the communication medium, we redesign the cryptographic algorithms.

Classical Network + New Mathematics

Post-Quantum Cryptography (PQC)

- Runs on existing computers.
- Uses existing Internet infrastructure.
- Resists attacks from both classical and quantum computers.
- Requires no quantum communication channel.

Modern standards replace Diffie-Hellman by **Post-Quantum Key Encapsulation Mechanisms (KEMs)**.

Hardness Assumptions for Post-Quantum Cryptography

Unlike classical cryptography, PQC relies on mathematical problems for which no efficient classical or quantum algorithms are currently known.

Family	Representative Assumption
Lattice-based	LWE, MLWE, SIS
Code-based	Syndrome Decoding
Multivariate	MQ Problem
Hash-based	Hash Function Security
Isogeny-based	Isogeny Problems

Today

The NIST post-quantum standards primarily rely on **lattice-based cryptography**.

What If a Hardness Assumption Is Broken?

Suppose a future breakthrough shows that a widely used hardness assumption (e.g., lattice problems) can be solved efficiently.

Immediate Consequence

All cryptographic primitives whose security relies on that assumption would need to be replaced.

Fortunately, post-quantum cryptography is based on multiple independent families of hard problems.

Family	Example
Code-based	McEliece
Hash-based	XMSS, SPHINCS ⁺
Multivariate	MQ-based Schemes
Isogeny-based	SIKE (Research)
Lattice-based	ML-KEM, ML-DSA

Future Directions

- Develop cryptography from new mathematical assumptions.
- Explore alternative computational paradigms (e.g., DNA computing, molecular cryptography, etc.).
- Continuously analyze, standardize, and diversify cryptographic assumptions.

Next Topic

Next Topic

Public Key Encryption: and its usage in Digital Signatures.

Textbook:

- 1 Douglas R. Stinson and Maura Paterson. *Cryptography: Theory and Practice*. 4th Edition. Chapman & Hall/CRC Press, 2019. ISBN: 9780367148591.

Acknowledgment:

- The figures in this work were generated with the assistance of ChatGPT (OpenAI).



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.